Ajout d'un rendez-vous

1. Principe général

L'application C# va mettre à jour la base de données 'gsb' en utilisant exclusivement des procédures stockées

La question principale concerne l'emplacement des contrôles des règles métier.

Ces derniers peuvent être placés dans la procédure stockées ou dans un trigger.

Avantage de la procédure

Plus performante (Code précompilé)
Plus facilement testable

Inconvénient

Possibilité d'enregistrer directement des données sans contrôle.
Ne contrôle qu'un seul point d'entrée

Bref en termes de sécurité, c'est une solution à éviter sauf si elles sont le seul point d'accès et que l'accès direct aux tables est interdit ce qui en soi est peu réaliste.

Quand utiliser obligatoirement des triggers ?

Quand plusieurs applications écrivent dans la base
Quand des imports CSV existent
Lorsque des insertions directes sont possibles
Lorsqu'on ne maîtrise pas tous les points d'entrée

Le trigger est toujours la barrière de sécurité ultime.

Quels sont les inconvénients d'un trigger

Le trigger se déclenchant automatiquement la logique devient invisible et donc difficile à comprendre.
Il est plus difficile de trouver l'origine de l'erreur si on ne connait pas le code du trigger.
Si une règle change, il faut penser à adapter le trigger.
De plus un trigger peut en cacher un autre (effets secondaires).
Il est plus difficile à tester qu'une procédure
Il peut être légèrement moins performant.

Conclusion : On opte pour la mise en place des contrôles au niveau des triggers, les procédures se limiterons à lancer l'instruction de mise à jour.

A savoir : il est toujours possible d'ajouter des contrôles dans les formulaires se basant sur l'objet Service afin d'éviter un appel de la procédure stockée.

2. Le trigger avantAjoutVisite

Lorsqu'on développe une application de gestion, il ne suffit pas de contrôler les données côté interface. Il est essentiel de garantir l'intégrité directement au niveau de la base de données. En effet, plusieurs applications ou utilisateurs peuvent accéder aux mêmes tables : la base doit donc être capable de faire respecter elle-même les règles métier.

Avant d'insérer une nouvelle ligne dans la table Visite, plusieurs règles métier doivent être respectées :

- Le praticien doit appartenir au secteur (département) du visiteur.
- Le motif de visite doit exister.
- La visite ne peut pas être programmée :
  - dans moins d'une heure,
  - à plus de 2 mois,
  - un dimanche,
  - en dehors de la plage 8h–19h.
- Aucun médicament ni bilan ne doit être renseigné lors de la programmation.
- Il doit y avoir au moins 2 heures entre deux visites d'un même visiteur.
- Une seule visite non clôturée par praticien est autorisée.

1. Ouvrir sous PhpStorm le projet PHP : Database – Fichier : Sql>Trigger>avantAjoutVisite.sql et compléter le trigger avantAjoutVisite

3. La procédure ajouterRendezVous

Elle lance une requête insert sur la table visite en fournissant les valeurs sur les champs idVisiteur, idPraticien, idMotif, et dateEtHeure.

Les trois derniers sont transmis en paramètre à la procédure, le premier est récupéré depuis la vue leVisiteur.

Nous devons ici récupérer la valeur du champ id de type auto-increment. Cet identifiant sera utilisé dans l'application pour créer l'objet Visite qui sera ajouté dans la collection mesVisites de l'objet Session

Pour cela plusieurs 3 solutions sont possibles :

Solution A : Ajouter un quatrième paramètre en sortie afin de retourner l'identifiant de la nouvelle visite du type auto-increment.

Pour récupérer la valeur de l'auto-incrément, il existe plusieurs solutions :
select @@IDENTITY : retourne la valeur du dernier auto-increment mise à jour (déconseillé)
LAST_INSERT_ID() : retourne la dernière valeur générée dans la session courante (fiable)
Extrait de l'appel dans la méthode ajouterRendezVous de la classe Passerelle

```
MySqlParameter paramSortie = new MySqlParameter("_idVisite", MySqlDbType.Int32);
paramSortie.Direction = ParameterDirection.Output;
cmd.Parameters.Add(paramSortie);

cmd.ExecuteNonQuery();
return (int)paramSortie.Value!;
```

Solution B : Ajouter à la suite de la requête insert, select *LAST_INSERT_ID*() as id;
Extrait de l'appel dans la méthode ajouterRendezVous de la classe Passerelle

```
return Convert.ToInt32(cmd.ExecuteScalar());
```

Solution C : Ne rien ajouter dans la procédure qui réalise juste la requête insert et utiliser coté C#, la propriété LastInsertedId de l'objet MySqlCommand cmd

Extrait de l'appel dans la méthode ajouterRendezVous de la classe Passerelle

```
cmd.ExecuteNonQuery();
return (int) cmd.LastInsertedId;
```

2. Compléter la procédure stockée ajouterRendezVous (fichier Sql>Procédure>ajouterRendezVous.sql)

Rappel : La clause SQL SECURITY indique le compte utilisé pour exécuter la procédure : definer représente l'utilisateur ayant créé la procédure, invoker représente l'utilisateur courant.
La valeur par défaut est definer.

4. La méthode ajouterRendezVous de la classe passerelle

La classe Passerelle est une couche d'accès aux données (DAL). Son rôle consiste à exécuter des requêtes Elle ne doit pas traiter les exceptions mais éventuellement les capturer (try), et les laisser remonter (throw) vers le contrôleur (ici le formulaire de saisie).

Deux possibilités existent pour transmettre les paramètres à l'objet MySqlCommand

```
cmd.Parameters.AddWithValue("_dateEtHeure", uneDate);
```

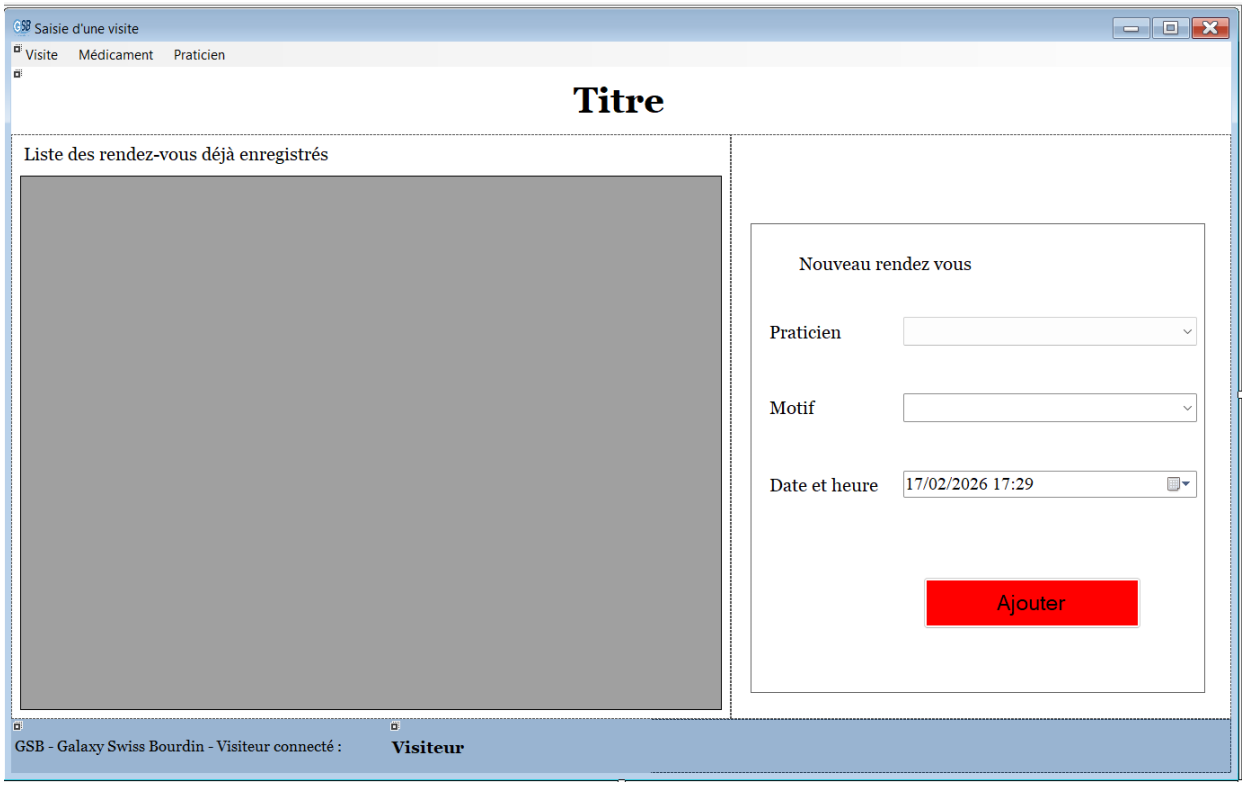
```
cmd.Parameters.Add("_dateEtHeure", MySqlDbType.DateTime).Value = dateHeure;
```

La seconde solution est plus sécurisée.

3. Compléter la méthode ajouterRendezVous de la classe Passerelle en fonction de la solution choisie à la question précédente

5. Construction de l'interface

Il faut commencer par placer les panels qui vont permettre d'adapter le positionnement des éléments en fonction de la taille du formulaire
L'ordre de création des « panel » est très important !
panelDroite pour la saisie avec dock : left
panelGauche pour le datagridview et le label : dock : fill



On commence par la partie Ajout (PanelDroite) **en dock Rigth**

Ajout à l'intérieur de PanelDroite, d'un panel pour contenir l'ensemble des composants graphiques afin de faciliter leur déplacement
Dans ce panel

- 4 labels non renommés
- 2 comboBox : **cbxPraticien** et **cbxMotif**
- 1 DateTimePicker **dtpDate**
- 1 button btnAjouter color : red

Le panelGauche en Dock : fill avec **Padding-All : 10** pour éviter que les composants internes soient collés

- 1 label 'Liste des rendez-vous déjà enregistrés' en **Dock : top** et **AutoSize : false**
- 1 datagridview **dgvVisites** en Dock : fill

6. Le code du formulaire

Rappel : Ne pas oublier de réorganiser le code dans deux régions : 'Procédures événementielles' et 'méthodes'

4. Recopier la méthode parametrerDgv depuis le fichier fourni '03 parametrerDgv.txt'

5. Ajouter et compléter la méthode remplirDgv

```
private void remplirDgv() {
   // vider le datagridView

   // Parcourir les visites sans bilan dans l'ordre chrnologique pour les ajouter dans le datagridview

}
```

6. Ajouter et compléter la méthode parametrerComposant()

```
      private void parametrerComposant()  {
         this.lblTitre.Text = "Enregistrer un nouveau rendez-vous";

         // alimentation de la zone de liste déroulante contenant les praticiens

         // alimentation de la zone de liste déroulante contenant les motifs

         parametrerDgv(dgvVisites);
         remplirDgV();
      }
```

7. Ajouter et compléter la procédure événementielle Load

Pour ajouter une procédure événementielle il faut partir du formulaire en mode conception et de double cliquer dans les propriétés sur l'événement à programmer (ici Load). La méthode événementielle sera alors automatiquement ajoutée dans le code (à la fin)

Pensez immédiatement à la déplacer dans la région 'Procédures événementielles'

```
private void FrmVisiteAjout_Load(object sender, EventArgs e) {
   parametrerComposant();
   remplirdgvVisites();
}
```

8. Tester l'application afin de vérifier l'affichage des rendez-vous dans le dataGridView

9. Ajouter et compléter la méthode ajout()

```
private void ajout()  {
    // vérifier la date et l'heure >= dans une heure


    // vérifier si c'est un dimanche

    // Vérifier l'heure entre 8 h et 19 h

    // vérifier les conflits de rendez-vous pour le praticien sélectionné


    try   {
        // récupérer les données saisies


        // enregistrer dans la base de données

        // créer la visite et l'ajouter à la session

        // mettre à jour le datagridview

        // message de confirmation

        // activer l'option du menu permettant la modification de la visite dans le menu

    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
 }
```

10. Ajouter et compléter la procédure événementielle chargée de gérer le clic sur le bouton btnAjouter

```
private void btnAjouter_Click(object sender, EventArgs e) {
    ajout();
}
```

11. Tester l'application afin de vérifier son fonctionnement.

12. Mettre à jour votre dépôt GitHub à partir de Visual Studio ou à l'aide des commandes git